Quantum Simulation Lab

Experiment I
Quantum Teleportation

Name: Hrishikesh M Jayakrishnan ID: SC26D054

---

1. Aim

1. To implement and demonstrate the quantum teleportation protocol using a quantum circuit.
2. To verify the successful transfer of an arbitrary quantum state from a sender to a receiver using the Qiskit framework.

2. Resources used

- Python
- Qiskit
- Jupyter Notebook environment

3. Theoretical Background

3.1 Quantum Teleportation

Quantum teleportation is a process by which quantum information (e.g., the exact state of an atom or photon) can be transmitted from one location to another, with the help of classical communication and previously shared quantum entanglement between the sending and receiving locations.

The protocol uses three qubits:

- **Qubit C:** The state to be teleported, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$.
- **Qubit A** and **Qubit B:** An entangled pair of qubits (Bell state) shared between the sender (Alice) and the receiver (Bob).

3.2 Protocol Steps

1. **Entanglement Generation:** A Bell state is created between Qubit A and Qubit B.

2. **Bell State Measurement:** Alice performs a CNOT gate on Qubit A controlled by Qubit C, followed by a Hadamard gate on Qubit C. She then measures both Qubits C and A.

3. **Classical Communication:** Alice sends the two classical bits obtained from her measurements to Bob.

4. **Conditional Operations:** Based on the classical bits received, Bob applies appropriate Pauli gates (X and/or Z) to his Qubit B to recover the original state $|\psi\rangle$.

4. Quantum circuit

4.1 Circuit Setup

The experiment is conducted following these steps in the Qiskit framework:

1. Initialize the quantum circuit with 3 quantum registers and 2 classical registers for Alice's measurements.

2. Prepare the initial random quantum state $|\psi\rangle$ on the first qubit.

3. Apply a Hadamard gate and a CNOT gate to the second and third qubits to create the entangled Bell pair.

4. Entangle the unknown state $|\psi\rangle$ to the first qubit and apply hadamard gate to it.

5. Construct the teleportation protocol by applying Alice's operations and Bob's conditional operations based on measurement outcomes.

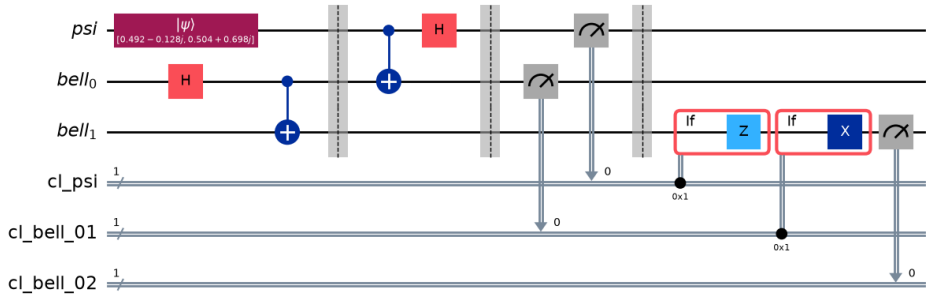


Figure 1: Quantum Teleportation Circuit

5. Measurement Results and Analysis

The circuit is executed on the Qiskit Aer Simulator for 10000 shots, and the results obtained are shown in the figures below.

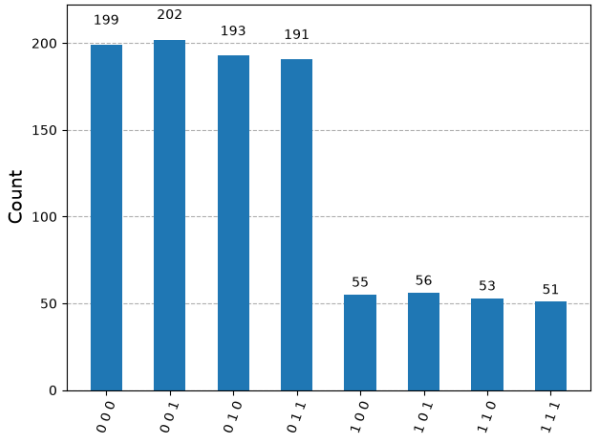


Figure 2: Measurement results for the quantum teleportation protocol

The results show two clear groups, with $000-011$ occurring much more frequently than $100-111$ (21.5%), indicating Bob's final measurement is in state $|0\rangle$.

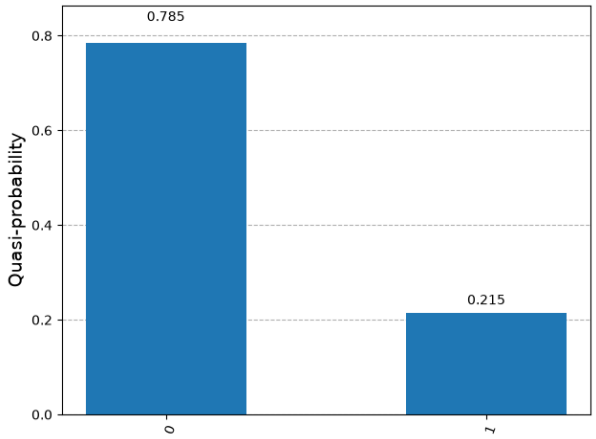


Figure 3: Probabilities of Bob's final state

The receiver measurement shows probabilities of approximately $P(0) \approx 0.785$ and $P(1) \approx 0.215$, indicating that Bob's teleported state is measured as $|0\rangle$ in about 78.5% of the shots and $|1\rangle$ in about 21.5% of the shots.

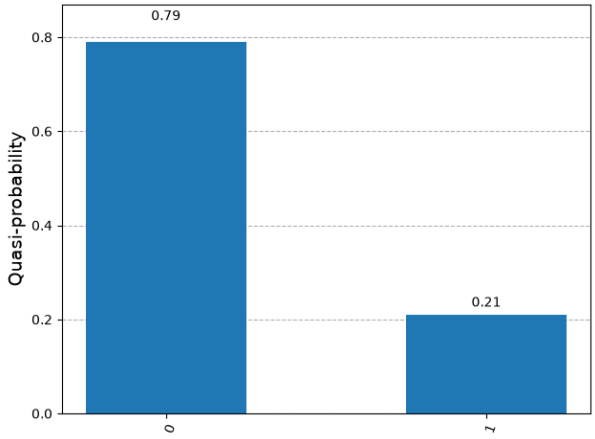


Figure 4: Probabilities of the initial state $|\psi\rangle$

The input-state measurement shows probabilities of approximately $P(0) \approx 0.79$ and $P(1) \approx 0.21$, indicating that the original $|\psi\rangle$ state is measured as $|0\rangle$ in about 79% of the shots and $|1\rangle$ in about 21%.

6. Conclusion

The quantum teleportation protocol was successfully implemented. The visual and numerical results confirm that the arbitrary state $|\psi\rangle$ was accurately transferred from the source qubit to the target qubit.

7. Appendix

7.1 Python Code

```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
from qiskit.quantum_info.states.random import random_statevector
from qiskit.circuit.library import Initialize
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
import numpy as np

psi = QuantumRegister(1, "psi")
cl_psi = ClassicalRegister(1, "cl_psi")
bell = QuantumRegister(2, "bell")
cl_bell_01 = ClassicalRegister(1, "cl_bell_01")
cl_bell_02 = ClassicalRegister(1, "cl_bell_02")
qc = QuantumCircuit(psi, cl_psi, bell, cl_bell_01, cl_bell_02)
qc.draw("mpl")

random_vector = random_statevector(2)
probs = np.abs(random_vector)**2
probs = {0: probs[0], 1: probs[1]}
state = Initialize(random_vector)
qc.append(state, psi)

qc.h(bell[0])
qc.cx(bell[0], bell[1])
qc.barrier()
qc.cx(psi, bell[0])
qc.h(psi)
qc.barrier()
qt_sv = Statevector.from_circuit(qc)
qc.measure(bell[0], cl_bell_01)
qc.measure(psi, cl_psi)
qc.barrier()

with qc.if_test((cl_psi, True)):
qc.z(bell[1])

with qc.if_test((cl_bell_01, True)):
qc.x(bell[1])
qc.measure(bell[1], cl_bell_02)
qc.draw("mpl")

sim = AerSimulator()
result = sim.run(qc, shots = 1000).result()

plot_histogram(result.get_counts())

from collections import Counter
receiver_count = Counter()
```

```
for k, v in result.get_counts().items():
receiver_count[k[0]] += v/1000

plot_histogram(receiver_count)
plot_histogram(probs)
```